

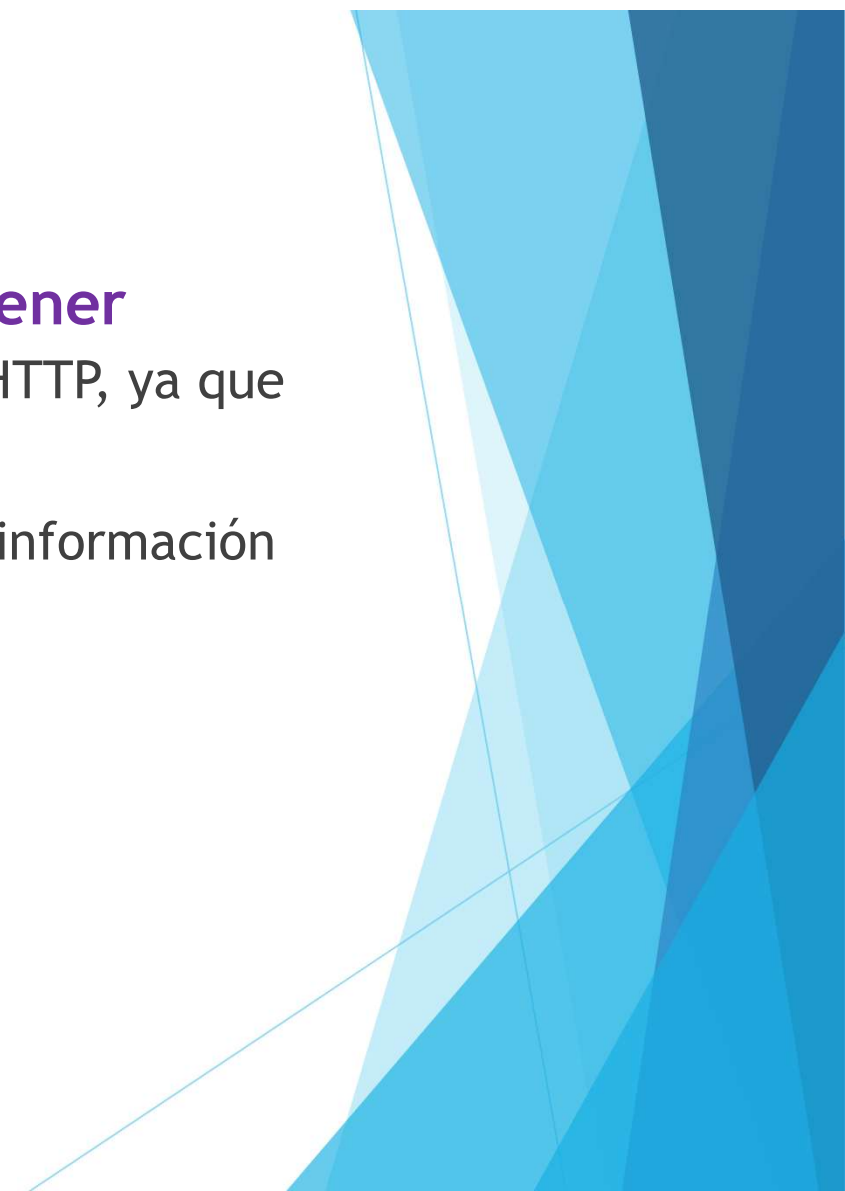
Sesiones y Cookies en Servlet



Introducción

En las aplicaciones Web, es fundamental **mantener información del usuario** entre solicitudes HTTP, ya que **el protocolo HTTP es sin estado**.

Las sesiones y cookies permiten mantener esta información entre solicitudes.



Sesiones

- ▶ **Definición:** Forma de almacenar información del usuario en el servidor mientras navega por la aplicación Web.
- ▶ **Duración:** Mientras la sesión permanezca activa o hasta que expire por inactividad.
- ▶ **Identificación:** Mediante un **session ID** único por usuario, normalmente almacenado en una cookie.
- ▶ **Ventajas:**
 - ▶ Los datos no se exponen al cliente.
 - ▶ Permiten almacenar objetos complejos.

Cookies

- ▶ **Definición:** Archivos pequeños de texto que el servidor envía al navegador y se reenvían con cada solicitud posterior.
- ▶ **Duración:**
 - ▶ *Cookies de sesión:* Se eliminan al cerrar el navegador.
 - ▶ *Cookies persistentes:* Se guardan en disco y sobreviven reinicios ya que tienen un tiempo de expiración.
- ▶ **Usos comunes:**
 - ▶ Almacenar preferencias.
 - ▶ Identificar usuarios.
 - ▶ Recordar datos entre visitas.
- ▶ **Precaución:** Los datos en cookies pueden ser manipulados por el cliente.

Relación entre sesiones y cookies

- ▶ El servidor crea una sesión y genera un session ID.
- ▶ Este ID se envía al navegador en una cookie.
- ▶ En solicitudes posteriores, la cookie permite al servidor identificar la sesión.
- ▶ Si la cookie no puede recuperarse, el servidor no podrá asociar correctamente la sesión del usuario.



Implementación de sesiones en un Servlet

- ▶ **Crear una sesión:** Obtiene o crea una sesión:
`HttpSession session = request.getSession();`
- ▶ **Almacenar datos en la sesión:**
`session.setAttribute("usuario", "mpastarini");`
- ▶ **Recuperar datos de la sesión:**
`String usuario = (String) session.getAttribute("usuario");`
- ▶ **Invalidar la sesión:**
`session.invalidate();`

Configuración en Web.xml

```
<session-config>  
  <session-timeout>30</session-timeout> <!-- minutos -->  
</session-config>
```

Uso de cookies en un Servlet

► Crear y enviar una cookie:

```
Cookie cookie = new Cookie("usuario", "mpastarini");  
cookie.setMaxAge(60*60*24); // Duración de 1 día  
cookie.setHttpOnly(true); // Protección contra XSS  
cookie.setSecure(true); // Para conexiones HTTPS  
cookie.setPath("/");  
response.addCookie(cookie);
```

NOTA 1: Si no se define tiempo de expiración se considera que es una cookie de sesión; será válida durante la sesión del navegador.

NOTA 2: **No guardar información sensible en cookies**, porque:

- el cliente puede modificarlas
- pueden ser leídas

Uso de cookies en un Servlet (Cont.)

► Leer cookies desde una solicitud:

```
Cookie[] cookies = request.getCookies();
String usuario = null;
if (cookies != null) {
    for (Cookie cookie : cookies) {
        if (cookie.getName().equals("usuario")) {
            usuario = cookie.getValue();
            break;
        }
    }
}
```

Uso de cookies en un Servlet (Cont.)

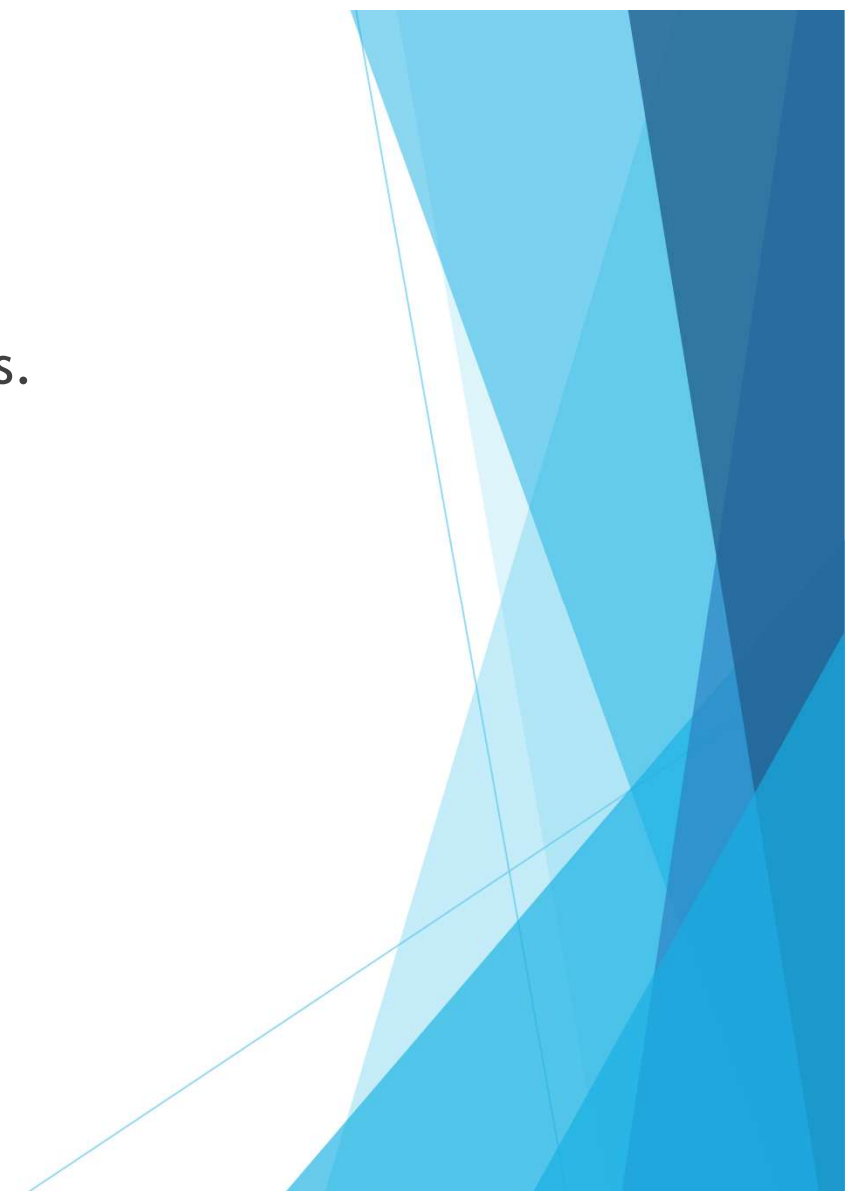
- **Eliminar una cookie:**

```
Cookie cookie = new Cookie("usuario", "");  
cookie.setMaxAge(0); // Expira la cookie  
cookie.setPath("/");  
response.addCookie(cookie);
```

Seguridad en Sesiones y Cookies

Buenas prácticas:

- ▶ Usar HttpOnly y Secure para cookies sensibles.
- ▶ Regenerar session ID tras inicio de sesión.
- ▶ Invalidar sesión en logout.
- ▶ Establecer timeout adecuado.



Seguridad en Sesiones y Cookies (cont.)

Riesgos:

- ▶ **Secuestro de sesión:** cuando un atacante roba el identificador de sesión del usuario y logra hacerse pasar por él.
- ▶ **Robo de cookies:** si las cookies no están protegidas correctamente, pueden ser interceptadas o robadas.
- ▶ **Inyección de scripts (XSS):** Si la aplicación permite insertar scripts maliciosos, un atacante podría acceder a cookies o información del usuario mediante JavaScript.

